Universidad Icesi

Filtros, SecurityFilterChain y Seguridad Stateless con JWT

Sesión 2: OncePerRequestFilter, Estándar RFC 7519 e Implementación con JJWT 0.12+

Computación en Internet II — Universidad Icesi

Ruta de Aprendizaje — 4 Micro-Ciclos (Teoría ↔ Práctica)

Primera Mitad: Estándar JWT y Flujo de Login

01. Stateful vs Stateless & Estándar RFC 7519

Anatomía Header.Payload.Signature, Base64URL vs Encriptación + Dependencias `JJWT 0.12.6`.

02. Flujo de Autenticación, Login & Emisión

DTOs, `JwtService.generateToken()`, `AuthRestController` y prueba inmediata en Postman + `jwt.io`.

Enfoque Práctico: Primero entendemos cómo nace el token (Login) antes de construir la capa de intercepción que lo consume.

Segunda Mitad: Autorización, Filtros y Seguridad Dual

03. Filtros, Dual Chain & RBAC (@PreAuthorize)

Verificación criptográfica, `JwtAuthenticationFilter` y coexistencia `@Order(1)` API + `@Order(2)` MVC.

04. Escenarios en Producción & Laboratorio Final

Robo de token, sabotaje de firma (`401`), Stale Claims y batería completa de 5 pruebas en Postman.

Meta Final: Proteger los endpoints bajo `/api/**` de forma Stateless sin romper las vistas Thymeleaf MVC.

Universidad ICESI

01. Arquitectura Stateless y Anatomía de JWT (RFC 7519)

De la cookie JSESSIONID en memoria RAM al token autoportante firmado criptográficamente.

Contraste Arquitectónico: Sesiones Stateful vs. Tokens Stateless (JWT)

Estado en Memoria del Servidor vs. Identidad Autoportante

Por qué las APIs RESTful reemplazan la HttpSession en RAM por tokens firmados digitalmente

MODELO STATEFUL (MVC)

1. Sesión Almacenada en Servidor

Transporte: **Cookie: JSESSIONID=8A5C1E...**

```
TOMCAT MEMORY (HttpSession Map)
Key: "8A5C1E9D" ➔ SecurityContext
Principal: UserDetails (admin)
Problema: Consume RAM por cada usuario
```

LIMITACIONES EN ARQUITECTURAS REST

- **Escalamiento Horizontal:** Si un balanceador envía el request al Nodo B, ese nodo no tiene la sesión en RAM.
- **Acoplamiento a Cookies:** Exige soporte de cookies y protección adicional contra ataques CSRF.

MODELO STATELESS (REST)

2. Token Autoportante (JWT)

Transporte: **Authorization: Bearer <JWT>**

```
VERIFICACIÓN MATEMÁTICA EN CPU
Jwts.parser().verifyWith(secretKey)
0 Bytes de HttpSession en Memoria RAM
SessionCreationPolicy.STATELESS
```

VENTAJAS CLOUD-NATIVE Y MOBILE

- **100% Escalable:** Cualquier instancia del servidor valida el token usando la clave secreta compartida.
- **Clientes Nativos:** Ideal para apps móviles, SPAs y comunicación entre microservicios.

Anatomía de un JSON Web Token (RFC 7519): Header . Payload . Signature

Estructura Compacta Separada por Puntos (xxxxx.yyyyy.zzzzz)

Tres segmentos codificados en Base64URL que viajan en la cabecera HTTP Authorization: Bearer



1. HEADER (METADATOS)

`{"alg": "HS256", "typ": "JWT"}`

- **alg:** Algoritmo criptográfico de firma (HMAC-SHA256 o RSA).
- **typ:** Declara el estándar del token (JSON Web Token).

100% legible al decodificar Base64.

2. PAYLOAD (CLAIMS)

`sub, iat, exp + authorities`

- **sub (Subject):** Correo del usuario.
- **iat / exp:** Timestamps Unix de emisión y expiración obligatoria.
- **Custom Claims:** Lista de roles y permisos (USER_READ, etc.).

3. SIGNATURE (INTEGRIDAD)

`HMACSHA256(H + "." + P, key)`

- Se calcula combinando el Header, el Payload y la **clave secreta**.
- Impide que un atacante modifique su rol o extienda la fecha exp sin ser detectado al instante.

Regla de Oro: Un JWT estándar (JWS) está **FIRMADO, NO ENCRIPTADO** . Jamás guardes contraseñas ni datos privados en el Payload.

Universidad ICESI

Tres Conceptos Distintos en un JWT: Base64URL, Firma e Integridad

1. BASE64URL (TRANSPORTE)

Reversible sin Clave
El Header y el Payload se codifican en `Base64URL` únicamente para poder viajar como texto seguro en cabeceras HTTP.

Cualquiera puede leerlos en jwt.io.

2. FIRMA HMAC (INTEGRIDAD)

Inmutable sin Secreto
La tercera parte del token es un resumen criptográfico calculado con `jwt.secret`.

Garantiza que ni un solo bit del Payload fue alterado.

3. ¿QUÉ GUARDAR EN CLAIMS?

Solo Identidad y Vigencia
Guarda únicamente el identificador (`sub: email`), tiempos (`iat` , `exp`) y permisos públicos.

Nunca incluyas hashes ni secretos.

Algoritmos de Firma: Simétrico (HS256) vs. Asimétrico (RS256)

HS256 — Firma Simétrica (Clave Secreta Única)

Utiliza una **única clave secreta** (mínimo 256 bits / 32 bytes) tanto para **firmar** el token en el login como para **verificarlo** en cada petición entrante.

Velocidad y Simplicidad
Cálculo ultrarrápido en CPU. Ideal cuando el mismo backend Spring Boot emite y valida sus propios tokens.

RS256 — Firma Asimétrica (Clave Privada / Pública)

El Servidor de Identidad firma el JWT con su **Clave Privada** exclusiva, y los microservicios consumidores lo validan con la **Clave Pública**.

Aislamiento en Microservicios (OAuth2 / OIDC)
Los microservicios secundarios pueden verificar tokens sin poseer jamás la capacidad de emitir tokens nuevos.

Práctica Ciclo 1: Dependencias JJWT 0.12.6 y Propiedades

1. Dependencias Maven en pom.xml

```xml
<!-- 1. Contratos públicos de JJWT (Compilación) -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
</dependency>
<!-- 2. Implementación criptográfica (Runtime) -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
<!-- 3. Serializador JSON Jackson (Runtime) -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
```

2. Configuración en application.properties

```properties
# Clave secreta en Base64 (Mínimo 256 bits / 32 bytes)
jwt.secret=NTRmZjg5YzI4NzRkNDllY2I5YzA3ZjM4ZTVhMWI2YzRk...

# Tiempo de expiración en milisegundos (1 hora)
jwt.expiration=3600000
```

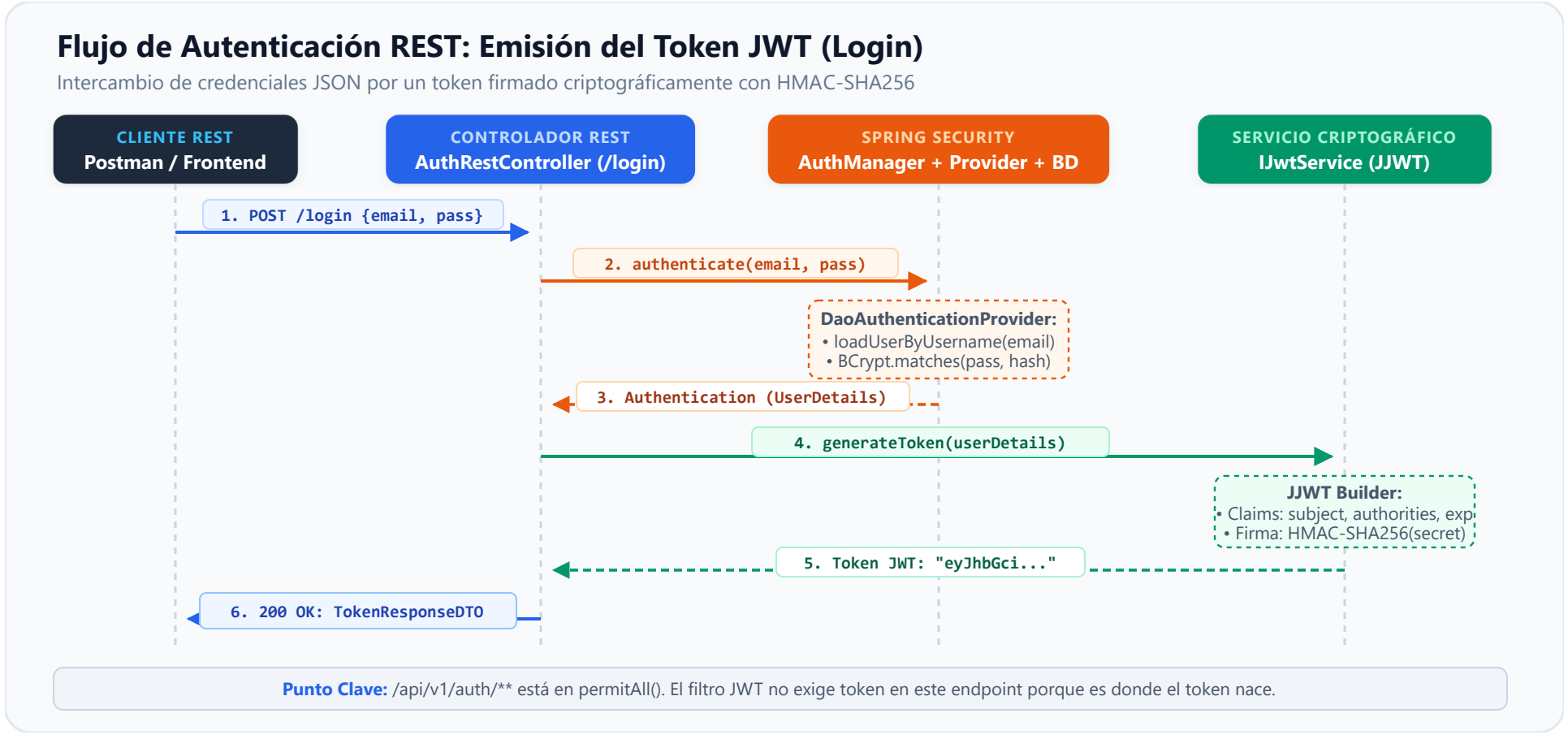
Protección contra Claves Débiles: `Keys.hmacShaKeyFor(bytes)` valida que la clave tenga al menos 256 bits. Si es menor, lanza `WeakKeyException` en el arranque.

Universidad ICESI

02. Flujo de Autenticación, Emisión de Tokens y Login REST

De las credenciales de usuario a la validación con BCrypt y la firma de un token JWT.

Arquitectura del Flujo de Autenticación: ¿Cómo Nace un Token JWT?



Práctica Ciclo 2 (Parte A): DTOs de Login y generateToken()

1. DTOs de Login y Respuesta de Token

```java
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class LoginRequestDTO {
    private String correoInstitucional;
    private String password;
}

@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class TokenResponseDTO {
    private String token;
    private String type;        // "Bearer"
    private long expiresIn;     // 3600000
    private String correoInstitucional;
}
```

Desacoplamiento: Los DTOs aíslan la entidad interna `Usuario` del contrato público JSON del endpoint REST.

2. generateToken() en JwtServiceImpl.java

```java
@Service
public class JwtServiceImpl implements IJwtService {

  @Value("${jwt.secret}")
  private String secretKeyBase64;

  @Value("${jwt.expiration}")
  private long jwtExpirationMs;

  @Override
  public String generateToken(UserDetails userDetails) {
    Map<String, Object> extraClaims = new HashMap<>();
    List<String> authorities = userDetails.getAuthorities().stream()
        .map(GrantedAuthority::getAuthority).toList();
    extraClaims.put("authorities", authorities);

    Date now = new Date(System.currentTimeMillis());
    return Jwts.builder()
        .claims(extraClaims)
        .subject(userDetails.getUsername())
        .issuedAt(now)
        .expiration(new Date(now.getTime() + jwtExpirationMs))
        .signWith(getSigningKey(), Jwts.SIG.HS256)
        .compact();
  }
```

Práctica Ciclo 2 (Parte B): AuthRestController y Verificación en Postman

1. controller/rest/AuthRestController.java

```java
@RestController
@RequestMapping("/api/v1/auth")
@RequiredArgsConstructor
public class AuthRestController {

  private final AuthenticationManager authManager;
  private final IJwtService jwtService;

  @PostMapping("/login")
  public ResponseEntity<TokenResponseDTO> login(
  @RequestBody LoginRequestDTO req) {
    try {
      Authentication auth = authManager.authenticate(
        new UsernamePasswordAuthenticationToken(
          req.getCorreoInstitucional(), req.getPassword()));
      UserDetails user = (UserDetails) auth.getPrincipal();
      String jwt = jwtService.generateToken(user);
      return ResponseEntity.ok(TokenResponseDTO.builder()
          .token(jwt).type("Bearer")
          .expiresIn(jwtService.getExpirationTime())
          .correoInstitucional(user.getUsername()).build());
    } catch (BadCredentialsException e) {
      return ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
    }
  }
}
```

2. Exponer Bean y Verificación en Postman + jwt.io

Exponer AuthenticationManager en SecurityConfig

`@Bean public AuthenticationManager authenticationManager(AuthenticationConfiguration c) throws Exception { return c.getAuthenticationManager(); }`

Test en Postman: POST /api/v1/auth/login

Enviar JSON con `admin@icesi.edu.co` / `admin123`. Recibimos `200 OK` con el token.

Inspección Inmediata en jwt.io

Pega el token en **jwt.io**: Comprueba que `sub` sea el email, `authorities` contenga los roles y la firma coincida.

Universidad ICESI

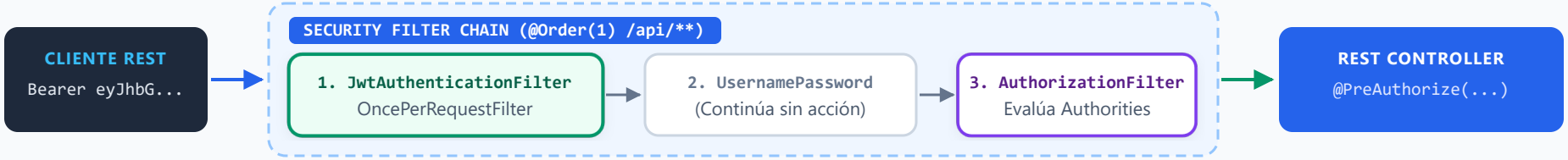
03. Flujo de Autorización: OncePerRequestFilter y SecurityFilterChain Dual

Una vez emitido el token: ¿cómo interceptar peticiones entrantes, verificar firmas y proteger los endpoints /api/**?

Arquitectura de Intercepción: JwtAuthenticationFilter en la Filter Chain

Flujo de Autenticación Stateless por Petición (.addFilterBefore)

El filtro JWT intercepta el header Authorization antes de UsernamePasswordAuthenticationFilter



PASO A: VALIDACIÓN CRIPTOGRÁFICA

IJwtService

1. Extrae el token tras `"Bearer "`.
2. Invoca `extractUsername(jwt)` verificando la firma HMAC-SHA256.
3. Comprueba que el token no haya expirado (`isTokenValid`).

Retorna: Subject (username)
Ej: "admin@icesi.edu.co"

PASO B: HIDRATACIÓN DE PERMISOS

UserDetailsService

1. Busca al usuario en BD mediante `loadUserByUsername(email)`.
2. Construye `UserDetails` con sus roles y permisos actualizados.
3. Verifica si la cuenta sigue activa.

GrantedAuthorities Frescos
ROLE_ADMIN, USER_READ, USER_DELETE

PASO C: INYECCIÓN EN EL HILO

SecurityContextHolder

1. Instancia un objeto autenticado: `UsernamePasswordAuthenticationToken`
2. Lo registra en el `ThreadLocal` de la petición actual.
3. Continúa con `filterChain.doFilter()`.

Petición Autenticada (OK)
Se destruye al terminar el Request

Universidad ICESI

OncePerRequest

1 Ejecución / Req

doFilterInternal

¿Por Qué Extender OncePerRequestFilter?

El Problema del Filter Estándar de Jakarta Servlet

En contenedores como Tomcat, un despacho interno (`RequestDispatcher.forward` o manejo de errores `/error`) puede provocar que un filtro normal se ejecute 2 o más veces en la misma petición HTTP.

Garantía de Ejecución Única por Request

`OncePerRequestFilter` registra un atributo de control en el request y garantiza que `doFilterInternal()` se ejecute **exactamente una sola vez** por cada petición del cliente.

Regla Fundamental de Continuidad

Si la petición no trae header `Authorization: Bearer` , el filtro **NO arroja error**: simplemente invoca `filterChain.doFilter(request, response)` para permitir rutas públicas como `/api/v1/auth/login` .

Práctica Ciclo 3 (Parte A): Verificación Criptográfica en JwtServiceImpl

```java
@Override
public String extractUsername(String token) {
  return extractClaim(token, Claims::getSubject);
}
@Override
public boolean isTokenValid(String token, UserDetails user) {
  final String username = extractUsername(token);
  return username.equals(user.getUsername()) && !isTokenExpired(token);
}
private boolean isTokenExpired(String token) {
  return extractClaim(token, Claims::getExpiration).before(new Date());
}
private Claims extractAllClaims(String token) {
  return Jwts.parser()
      .verifyWith(getSigningKey())
      .build()
      .parseSignedClaims(token)
      .getPayload();
}
private SecretKey getSigningKey() {
  byte[] keyBytes = Decoders.BASE64.decode(secretKeyBase64);
  return Keys.hmacShaKeyFor(keyBytes);
}
```

¿Qué Hace parseSignedClaims(token)?

1. Verificación de Firma (SignatureException)
Recalcula el hash HMAC-SHA256 con `getSigningKey()`. Si el payload fue modificado en 1 bit, lanza excepción.

2. Control de Vigencia (ExpiredJwtException)
Verifica automáticamente que la hora actual no supere el atributo `exp` del token.

3. Extracción Funcional con Function<Claims, T>
`extractClaim(token, Claims::getSubject)` aplica referencias a métodos para extraer cualquier atributo de forma tipada.

Universidad ICESI

Práctica Ciclo 3 (Parte B): Codificación de JwtAuthenticationFilter.java

1. Extracción de Cabecera Bearer y Token

```java
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

  private final IJwtService jwtService;
  private final UserDetailsService userDetailsService;

  @Override
  protected void doFilterInternal(
  @NonNull HttpServletRequest req,
  @NonNull HttpServletResponse res,
  @NonNull FilterChain chain)
      throws ServletException, IOException {

    final String authHeader = req.getHeader("Authorization");
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
      chain.doFilter(req, res);
      return;
    }
    final String jwt = authHeader.substring(7);
```

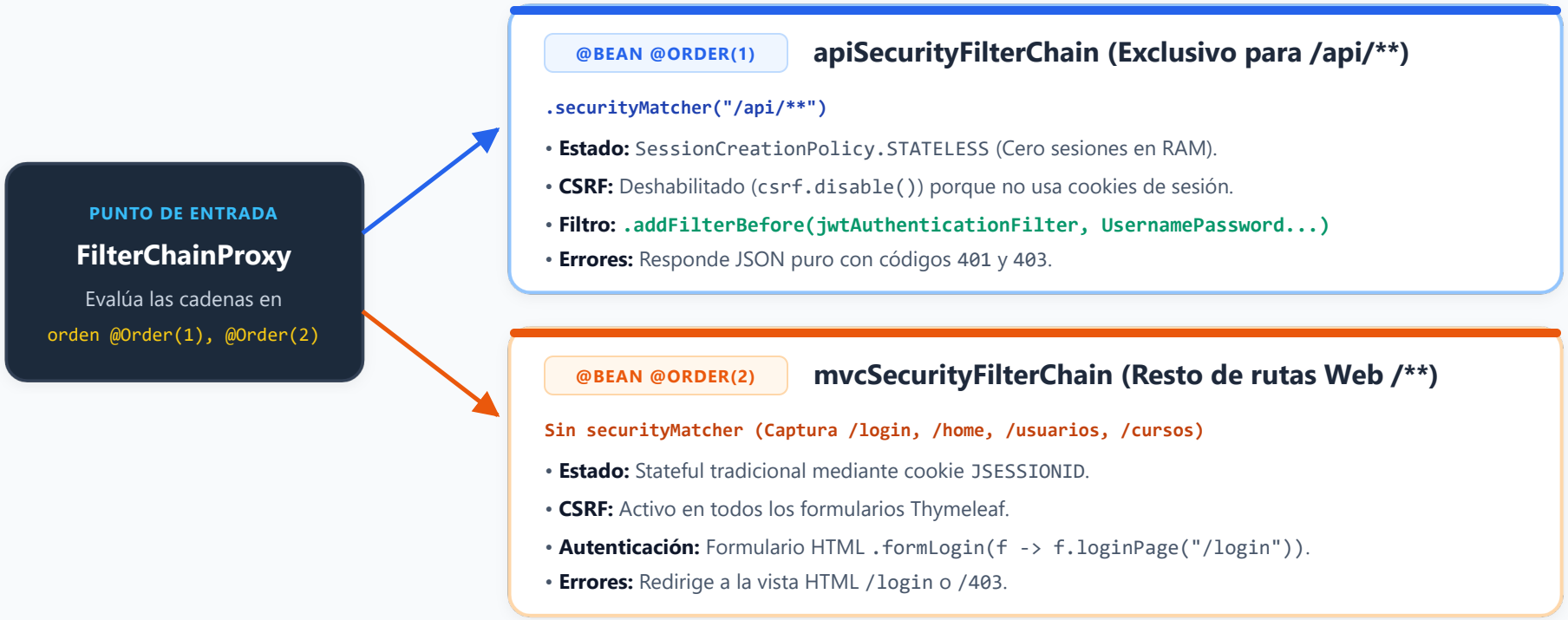
2. Validación e Inyección en SecurityContextHolder

```java
    try {
      final String email = jwtService.extractUsername(jwt);
      if (email != null && SecurityContextHolder.getContext()
          .getAuthentication() == null) {
        UserDetails user = userDetailsService
            .loadUserByUsername(email);
        if (jwtService.isTokenValid(jwt, user)) {
          var authToken = new UsernamePasswordAuthenticationToken(
              user, null, user.getAuthorities());
          authToken.setDetails(new WebAuthenticationDetailsSource()
              .buildDetails(req));
          SecurityContextHolder.getContext()
              .setAuthentication(authToken);
        }
      }
    } catch (JwtException | IllegalArgumentException e) {
      SecurityContextHolder.clearContext();
    }
    chain.doFilter(req, res);
  }
}
```

Coexistencia Arquitectónica: Dos SecurityFilterChain en el Mismo Proyecto

Enrutamiento Selectivo en FilterChainProxy Mediante @Order

Protegemos /api/** con JWT Stateless sin romper las vistas Thymeleaf MVC existentes



Práctica Ciclo 3 (Parte C): SecurityConfig.java con @Order(1)

```java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

  private final JwtAuthenticationFilter jwtAuthFilter;

  @Bean
  public AuthenticationManager authenticationManager(
  AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
  }

  @Bean
  @Order(1)
  public SecurityFilterChain apiFilterChain(HttpSecurity http)
      throws Exception {
    http
      .securityMatcher("/api/**")
      .csrf(csrf -> csrf.disable())
      .sessionManagement(sm -> sm
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/v1/auth/**").permitAll()
        .anyRequest().authenticated())
      .exceptionHandling(ex -> ex
        .authenticationEntryPoint((req, res, e) -> {
          res.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
          res.setContentType("application/json");
          res.getWriter().write(
            "{\"error\": \"Token JWT ausente o invalido\"}");
        })
        .accessDeniedHandler((req, res, e) -> {
          res.setStatus(HttpServletResponse.SC_FORBIDDEN);
          res.setContentType("application/json");
          res.getWriter().write(
            "{\"error\": \"Permisos insuficientes (403)\"}");
        })
      )
      .addFilterBefore(
        jwtAuthFilter,
        UsernamePasswordAuthenticationFilter.class
      );

    return http.build();
  }
  // @Bean @Order(2) mvcSecurityFilterChain(...) intacto
}
```

Universidad ICESI

Autorización Granular con @PreAuthorize y RBAC

Del "quién eres" al "qué puedes hacer": consumo de Authorities e inyección declarativa con SpEL.

Flujo de Autorización RBAC: De los Claims en el JWT a @PreAuthorize

Secuencia de Autorización Sin Consulta a Base de Datos (Zero-DB Hit)

De la extracción de claims en JwtAuthenticationFilter a la evaluación SpEL previa a cada método

PASO 1: TOKEN ENTRANTE

Payload de Claims

Cliente envía JWT en header:

`Authorization: Bearer ...`

```
// Claims del Token
{
 "sub": "admin@icesi",
 "authorities": [
  "ROLE_ADMIN",
  "USER_READ"
 ]
}
```

- Firma HMAC-SHA256 íntegra
- Claims auto-contenidos
- **Cero consulta a BD**

Capa: Cliente HTTP

PASO 2: FILTRO JWT

Hidratación Contexto

Parsea claims y crea las autoridades de Spring:

```
// Conversión a Authorities
SimpleGrantedAuthority:
 → ROLE_ADMIN
 → USER_READ

// Set en SecurityContext
SecurityContextHolder
.getContext().setAuth(...)
```

- ThreadLocal por petición
- Estado autenticado inmutable
- **Disponible para Spring AOP**

Capa: Security Filter

PASO 3: INTERCEPTOR AOP

@PreAuthorize SpEL

Proxy intercepta antes de invocar el método Java:

```
// Expresión en Controller
@PreAuthorize(
 "hasRole('ADMIN')"
 "or hasAuthority('READ')"
)

// Evaluación de Authorities
Coincide → Concede Acceso
```

- Seguridad declarativa limpia
- Desacoplado del controlador
- **Si no coincide: frena llamada**

Capa: Spring AOP Proxy

PASO 4: RESULTADO HTTP

Despacho Final

Respuesta HTTP según la evaluación de permisos:

200 OK **Autorizado**

Ejecuta el método Java y retorna DTO serializado.

403 FORBIDDEN **Denegado**

`AccessDeniedException`

Método Java nunca se invoca.

- Cero ejecución no autorizada
- Capturado por ExceptionHandler
- **JSON estructurado al cliente**

Capa: REST Controller

Habilitación Declarativa: Expresiones SpEL en @PreAuthorize

1. Roles (RBAC) vs. Permisos Atómicos

hasRole('ADMINISTRADOR')
Busca automáticamente la autoridad con prefijo `ROLE_ADMINISTRADOR`. Ideal para jerarquías amplias.

hasAuthority('USER_READ')
Evalúa el permiso directo sin asumir ningún prefijo. Permite un control de privilegios granular por acción.

2. Catálogo de Expresiones SpEL

Expresión SpEL	Significado y Uso
`hasRole('ADMIN')`	Requiere `ROLE_ADMIN`.
`hasAnyRole('A', 'B')`	Permite acceso si posee el rol A o el rol B.
`hasAuthority('READ')`	Exige el permiso puntual `READ`.
`hasRole('A') or hasAuthority('B')`	Combina roles con permisos usando operadores booleanos.
`isAuthenticated()`	Cualquier usuario autenticado legítimamente.

Universidad ICESI

Asegurando UsuarioRestController con @PreAuthorize

1. Endpoints de Consulta (Lectura Flexible)

```
@RestController
@RequestMapping("/api/v1/usuarios")
@RequiredArgsConstructor
public class UsuarioRestController {

    //Lectura permitida a Admin, Profesor o quien tenga USER_READ
    @GetMapping
    @PreAuthorize("hasAnyRole('ADMINISTRADOR', 'PROFESOR') " +
      "or hasAuthority('USER_READ')")
    public List<usuarioresponsedto> obtenerUsuarios(
    @RequestParam(required = false) Boolean active) {
        return usuarioMapper.toListUsuarioResponseDTO(
            usuarioService.listarUsuarios(active));
    }

    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole('ADMINISTRADOR', 'PROFESOR') " +
      "or hasAuthority('USER_READ')")
    public ResponseEntity<usuarioresponsedto> obtnerUsuarioPorId(
    @PathVariable Long id) {
        return usuarioService.obtenerPorId(id)
            .map(u -> ResponseEntity.ok(usuarioMapper.toUsuarioResponseDTO(u)))
            .orElse(ResponseEntity.notFound().build());
    }</usuarioresponsedto></usuarioresponsedto>
```

2. Endpoints de Mutación (Exclusivos Admin)

```
    //Creación, actualización y eliminación restringidas a ADMIN
    @PostMapping
    @PreAuthorize("hasRole('ADMINISTRADOR')")
    public ResponseEntity<usuarioresponsedto> crearUsuario(
    @RequestBody UsuarioRequestDTO dto) {
        //... creación con 201 Created ...
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasRole('ADMINISTRADOR')")
    public ResponseEntity<usuarioresponsedto> actualizarUsuario(
    @PathVariable Long id, @RequestBody UsuarioRequestDTO dto) {
        //... actualización ...
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMINISTRADOR')")
    public ResponseEntity<void> eliminarUsuario(@PathVariable Long id) {
        usuarioService.eliminarUsuario(id);
        return ResponseEntity.noContent().build();
    }
}</void></usuarioresponsedto></usuarioresponsedto>
```

04. Escenarios Críticos en Producción y Laboratorio Integral

Vulnerabilidades reales, robo de tokens, escalación de privilegios y validación exhaustiva de 5 pruebas.

Universidad ICESI

Tres Escenarios Críticos de Seguridad JWT en Producción

1. ROBO DE TOKEN VIGENTE

El Riesgo Stateless
Si un atacante roba un JWT válido (por XSS), el servidor lo aceptará hasta que llegue su fecha `exp`.

Mitigación: Emitir *Access Tokens* de vida corta (5–15 min) + *Refresh Tokens* en cookie `HttpOnly` o lista de revocación en Redis.

2. ALTERACIÓN DEL PAYLOAD

Escalación de Privilegios
Un estudiante decodifica su JWT en Base64, cambia `ROLE_ESTUDIANTE` por `ROLE_ADMINISTRADOR` y lo reenvía.

Defensa: Al no conocer la `jwt.secret`, el HMAC calculado falla (`SignatureException` ➔ `401 Unauthorized`).

3. STALE CLAIMS (DATOS VIEJOS)

Revocación de Rol en Vivo
¿Qué pasa si desactivamos un usuario cuyo token aún tiene 45 min de vida?

Defensa por Hidratación BD: Al consultar `loadUserByUsername(email)` en el filtro, obtenemos permisos frescos de la BD al instante.

Decisión de Arquitectura: Hidratación desde BD vs. 100% Claims del Token

Dimensión Arquitectónica	Enfoque A: Hidratación de UserDetails desde BD	Enfoque B: 100% Stateless Leyendo Claims del JWT
Costo en Base de Datos	1 consulta rápida por clave única (`username` / `email`)	**0 consultas SQL** (Solo verificación criptográfica en CPU)
Revocación de Roles / Permisos	**Efecto Inmediato** en la siguiente petición HTTP	Diferido hasta que el token expire (`exp`)
Desactivación de Cuenta (active=false)	**Bloqueo Inmediato** al evaluar el estado del usuario	El usuario sigue operando hasta que expire su JWT
Caso de Uso Recomendado	Sistemas académicos, financieros y empresariales críticos	Microservicios masivos con Access Tokens de 5 minutos

Balance de Ingeniería: Incluso consultando `UserDetailsService` , la arquitectura sigue siendo **100% Stateless en la capa web** porque el contenedor no almacena ningún objeto `HttpSession` en RAM entre peticiones.

Laboratorio Final en Vivo: 5 Pruebas en Postman y jwt.io

1. Petición sin Token ➔ 401 Unauthorized (JSON)

`GET /api/v1/usuarios` sin header `Authorization` responde 401 en JSON (ya no redirige al HTML `/login`).

2. Login con Rol Estudiante + Inspección en jwt.io

`POST /api/v1/auth/login` con credenciales de estudiante y decodificación de los *claims* en `jwt.io`.

3. Estudiante Invoca GET /usuarios ➔ 403 Forbidden

El token es auténtico, pero carece del permiso `USER_READ` exigido por `@PreAuthorize`.

4 y 5. Login Admin (200/201) y Sabotaje de Firma (401)

Autenticar con rol Administrador, ejecutar CRUD completo y alterar 1 carácter del token para verificar el rechazo criptográfico.

```
// POST /api/v1/auth/login
{
  "correoInstitucional": "admin@icesi.edu.co",
  "password": "admin123"
}

// Respuesta 200 OK:
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "type": "Bearer",
  "expiresIn": 3600000,
  "correoInstitucional": "admin@icesi.edu.co"
}
```

Universidad ICESI

Fin de la Sesión 2 — Semana 11

Seguridad Stateless con JWT, OncePerRequestFilter y Dual SecurityFilterChain